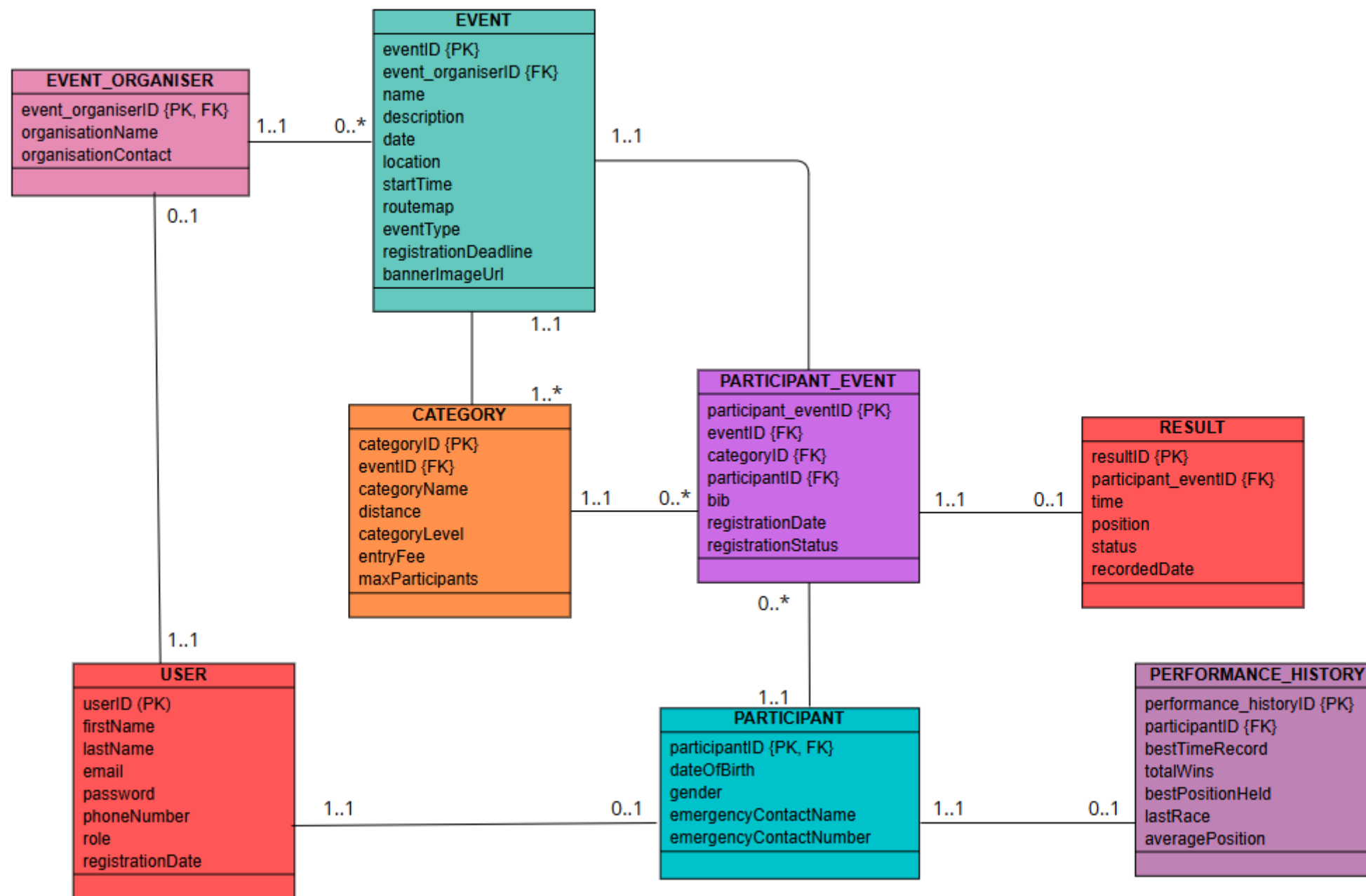




RaceDay ERD Explanation

The RaceDay Entity Relationship Diagram (ERD) defines the database structure used to manage user accounts, event organisers, participants, sporting events, event categories, registrations, race results, and participant performance history. The design separates general account information from role-specific information and uses **PARTICIPANT_EVENT** as an associative entity to connect participants to the specific events and categories in which they register. The entities, attributes, primary keys, foreign keys, relationships, and cardinalities are structured to represent how a real-world sports event platform would operate.

1. USER

The **USER** entity is the central account entity of the RaceDay system and provides the foundation for user authentication and account management. It contains **userID** (PK), **firstName**, **lastName**, **email**, **password**, **phoneNumber**, **role**, and **registrationDate**. The **userID** uniquely identifies each registered user, while **email** and **password** support authentication and **role** identifies the user's selected access role. The **registrationDate** records when the account was created. **USER** has a 1..1 to 0..1 relationship with **EVENT_ORGANISER**, meaning that every **EVENT_ORGANISER** profile must be associated with exactly one **USER**, while a **USER** may have zero or one organiser profile. **USER** also has a 1..1 to 0..1 relationship with **PARTICIPANT**, meaning that every **PARTICIPANT** profile must belong to exactly one **USER**, while a **USER** may have zero or one participant profile. This structure provides one central account for authentication while keeping organiser and participant information separate. In a real-world sports platform, a person would first create an account before accessing role-specific features such as organising events or registering for races.

2. EVENT_ORGANISER

The **EVENT_ORGANISER** entity stores information specific to users who organise and manage sporting events. It contains **event_organiserID** (PK, FK), **organisationName**, and **organisationContact**. The **event_organiserID** acts as both the primary key and foreign key because each organiser profile is directly linked to one **USER** account. **organisationName** identifies the organisation responsible for hosting an event, while **organisationContact** provides contact information for the organisation. **EVENT_ORGANISER** has a 1..1 to 0..* relationship with **EVENT**, meaning that every **EVENT** must be managed by exactly one **EVENT_ORGANISER**, while an organiser can manage zero, one, or many events. This reflects a realistic situation where a running club, charity, community organisation, or sports organisation can organise several events over time. Separating the organiser profile from **USER** allows the system to store organisation-specific information without mixing it with general account information.

3. EVENT

The **EVENT** entity represents the main sporting event managed through the RaceDay system. It contains **eventID** (PK), **event_organiserID** (FK), **name**, **description**, **date**, **location**, **startTime**, **routemap**, **eventType**, **registrationDeadline**, and **bannerImageUrl**. The **eventID** uniquely identifies each event, while **event_organiserID** identifies the organiser responsible for that event. The remaining attributes describe the event's name, purpose, date, location, starting time, route, type,

registration period, and visual representation on the website. The **eventType** can distinguish events such as marathons, fun runs, charity runs, or cycling events, while **registrationDeadline** indicates when registration closes and **bannerImageUrl** identifies the image used to represent the event on the website.

EVENT has a 1..1 to 1..* relationship with **CATEGORY**, meaning that every event must have at least one category, while every category belongs to exactly one event. This ensures that an event cannot exist without defining at least one participation option. **EVENT** also has a 1..1 to 0..* relationship with **PARTICIPANT_EVENT**, meaning that each registration belongs to exactly one event, while an event can have zero or many registrations. This allows an event to be created and displayed on the platform before any participants register. The **routemap** and **bannerImageUrl** are stored within **EVENT** because they describe the event itself rather than an individual participant or registration.

4. CATEGORY

The **CATEGORY** entity represents the different participation divisions available within a particular event. It contains **categoryID** (PK), **eventID** (FK), **categoryName**, **distance**, **categoryLevel**, **entryFee**, and **maxParticipants**. The **categoryID** uniquely identifies each category, while **eventID** links the category to its specific event. **categoryName** identifies the participation division, **distance** records the length of the race, and **categoryLevel** identifies the expected participation level, such as Beginner, Intermediate, or Advanced. **entryFee** stores the cost of entering the category, while **maxParticipants** defines the maximum number of participants allowed in that category.

CATEGORY belongs to exactly one **EVENT**, while each **EVENT** must have at least one **CATEGORY**, represented by the **EVENT (1..1) – CATEGORY (1..*)** relationship. **CATEGORY** also has a **1..1** to **0..*** relationship with **PARTICIPANT_EVENT**, meaning that every registration must select exactly one category, while a category can have zero or many registrations. Separating **CATEGORY** from **EVENT** allows one event to offer multiple participation options with different distances, levels, entry fees, and participant limits without placing all of these details directly in the **EVENT** entity.

5. PARTICIPANT

The **PARTICIPANT** entity stores information specific to users who take part in sporting events. It contains **participantID** (PK, FK), **dateOfBirth**, **gender**, **emergencyContactName**, and **emergencyContactNumber**. The **participantID** acts as both the primary key and foreign key because each participant profile is directly connected to one **USER** account. The remaining attributes provide information relevant to participation and participant safety, particularly the emergency contact details required during sporting events.

PARTICIPANT has a **1..1** to **0..*** relationship with **PARTICIPANT_EVENT**, meaning that every registration belongs to exactly one participant, while a participant can register for zero or many events. **PARTICIPANT** also has a **1..1**

to **0..1** relationship with **PERFORMANCE_HISTORY**, meaning that a participant can have zero or one performance-history record, while each performance-history record belongs to exactly one participant. A participant may have no performance history before completing an event, after which the system can maintain an aggregated summary of their previous performance. Separating **PARTICIPANT** from **USER** allows the system to store participant-specific information without placing all role-specific information into the general account entity.

6. PARTICIPANT_EVENT

The **PARTICIPANT_EVENT** entity is an associative entity that connects **PARTICIPANT**, **EVENT**, and **CATEGORY**. It is required because a participant can register for multiple events, while an event can have multiple participants, creating a many-to-many relationship that needs to be resolved through a separate registration entity. It contains **participant_eventID** (PK), **eventID** (FK), **categoryID** (FK), **participantID** (FK), **bib**, **registrationDate**, and **registrationStatus**.

The **participant_eventID** uniquely identifies each registration, while the three foreign keys identify the participant, event, and category associated with that registration. The **bib** stores the race identification number assigned to the participant for that particular event, while **registrationDate** records when the participant registered and **registrationStatus** records the current state of the registration, such as pending, confirmed, cancelled, or withdrawn. **PARTICIPANT_EVENT** has a **1..1** relationship with **PARTICIPANT**, **EVENT**, and **CATEGORY**, meaning that every registration must belong to exactly one participant, one event, and one category. It also has a **1..1** to **0..1** relationship with **RESULT**, meaning that a registration may exist without a result before the event takes place, but can have one result once the participant's performance has been recorded. This entity therefore represents the specific real-world registration instance and stores information that belongs to that registration rather than to the participant or event generally.

7. RESULT

The **RESULT** entity stores the official outcome of a participant's registration after their event participation. It contains **resultID** (PK), **participant_eventID** (FK), **time**, **position**, **status**, and **recordedDate**. The **resultID** uniquely identifies each result, while **participant_eventID** identifies the specific registration for which the result was produced. The **time** records the participant's race time, **position** records their finishing position, **status** records the outcome of their participation, and **recordedDate** records when the result was entered into the system. The relationship is represented as **PARTICIPANT_EVENT (1..1) – RESULT (0..1)**. This means that every result must belong to exactly one participant registration, while a registration may have zero or one result. The optional side reflects the real-world process because a participant can register for an event before the race takes place and therefore may not have a result yet. Once the participant's performance has been recorded, that registration can have one corresponding **RESULT**. A result cannot exist independently because it must always be associated with a specific participant registration.

8. PERFORMANCE_HISTORY

The **PERFORMANCE_HISTORY** entity stores an aggregated summary of a participant's performance across their previous events. It contains **performance_historyID** (PK), **participantID** (FK), **bestTimeRecord**, **totalWins**, **bestPositionHeld**, **lastRace**, and **averagePosition**. The **performance_historyID** uniquely identifies the performance-history record, while **participantID** connects the history to the relevant participant. **bestTimeRecord** stores the participant's fastest recorded race time, **totalWins** records the number of times the participant achieved first position, **bestPositionHeld** records their best finishing position, **lastRace** records their most recent race, and **averagePosition** provides an overall indication of their average finishing position across their recorded results. **PERFORMANCE_HISTORY** has a **1..1** to **0..1** relationship with **PARTICIPANT**, meaning that a participant can have zero or one performance-history record, while each performance-history record belongs to exactly one participant. The performance history is an aggregated summary rather than a replacement for individual race results. Individual **time**, **position**, and **status** values remain stored in **RESULT**, while **PERFORMANCE_HISTORY** provides the participant with an overall view of their achievements and progress across multiple events.

Overall ERD Structure

Overall, the RaceDay ERD follows the real-world flow of the platform. **USER** provides the central account foundation, which connects to the role-specific **EVENT_ORGANISER** and **PARTICIPANT** entities. **EVENT_ORGANISER** manages **EVENT** records, while each **EVENT** must contain at least one **CATEGORY** defining the available participation options. Participant registrations are represented through **PARTICIPANT_EVENT**, which connects a participant to a specific event and category while storing registration-specific information such as their **bib**, registration date, and registration status. Once an event takes place, a **PARTICIPANT_EVENT** registration can receive one **RESULT** containing the participant's time, position, status, and recorded date. The participant's results can then contribute to their **PERFORMANCE_HISTORY**, which provides an aggregated view of their overall performance. The cardinalities ensure that mandatory relationships are enforced where required, while optional relationships reflect the real-world timing of activities, such as a registration existing before a result is recorded or a participant having no performance history before completing an event. This structure provides RaceDay with a clear and realistic database design in which each entity has a specific responsibility while working together to support account management, event organisation, category management, participant registration, race results, and performance tracking.